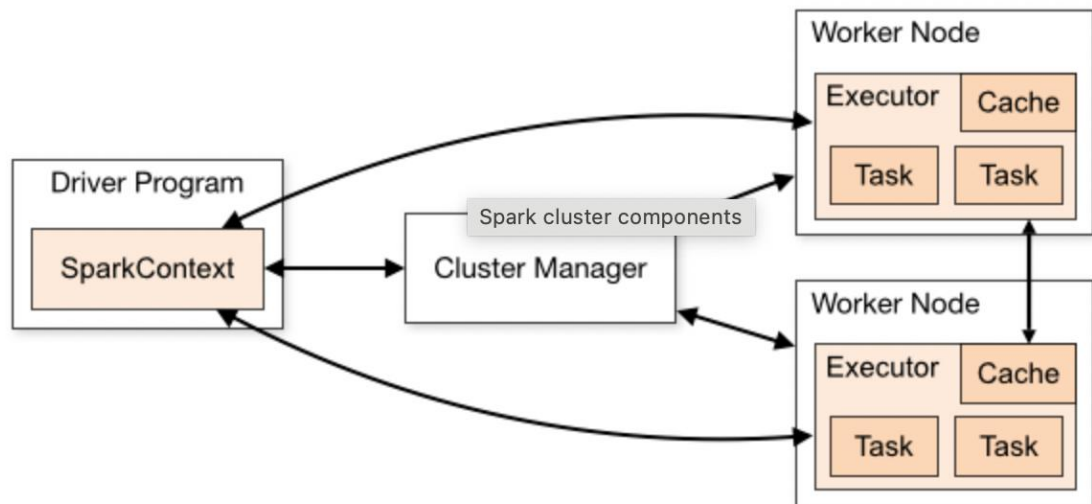


Week-6

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

- **Driver:** The main process that runs your application's `main()` function. It creates the `SparkContext/SparkSession`, converts your code into a DAG of tasks, schedules those tasks, and coordinates everything. It's the "brain" of the application.
- **Cluster Manager:** Allocates resources (CPU, memory) across the cluster. Examples: Standalone, YARN, Mesos, Kubernetes. It negotiates with the Driver to launch Executors on worker nodes.
- **Executor:** Worker processes launched on cluster nodes that actually execute the tasks assigned by the Driver, store data in memory/disk (caching), and report results/status back to the Driver.



Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Ans. Spark doesn't execute transformations (filter, select, map, etc.) immediately. Instead, it builds a logical execution plan (DAG) and only executes when an action (show, collect, count) is called. This helps because:

- Spark can optimize the entire chain at once (via Catalyst optimizer) — combining, reordering, or eliminating unnecessary steps.
- It avoids materializing intermediate results, saving memory and I/O.
- Enables optimizations like predicate/projection pushdown (e.g., filtering before reading unnecessary columns).

. Transformations (Building the Recipe)

When you apply a transformation like `filter()`, `map()`, or `groupBy()`, Spark does not process the rows. It acts like someone writing down a cooking recipe—no food is created yet. It builds a **Logical Plan** and maps the data dependencies via a [Directed Acyclic Graph \(DAG\)](#).

- **Examples:** `filter()`, `select()`, `join()`, `map()`, `withColumn()`.

2. Actions (Turning on the Stove)

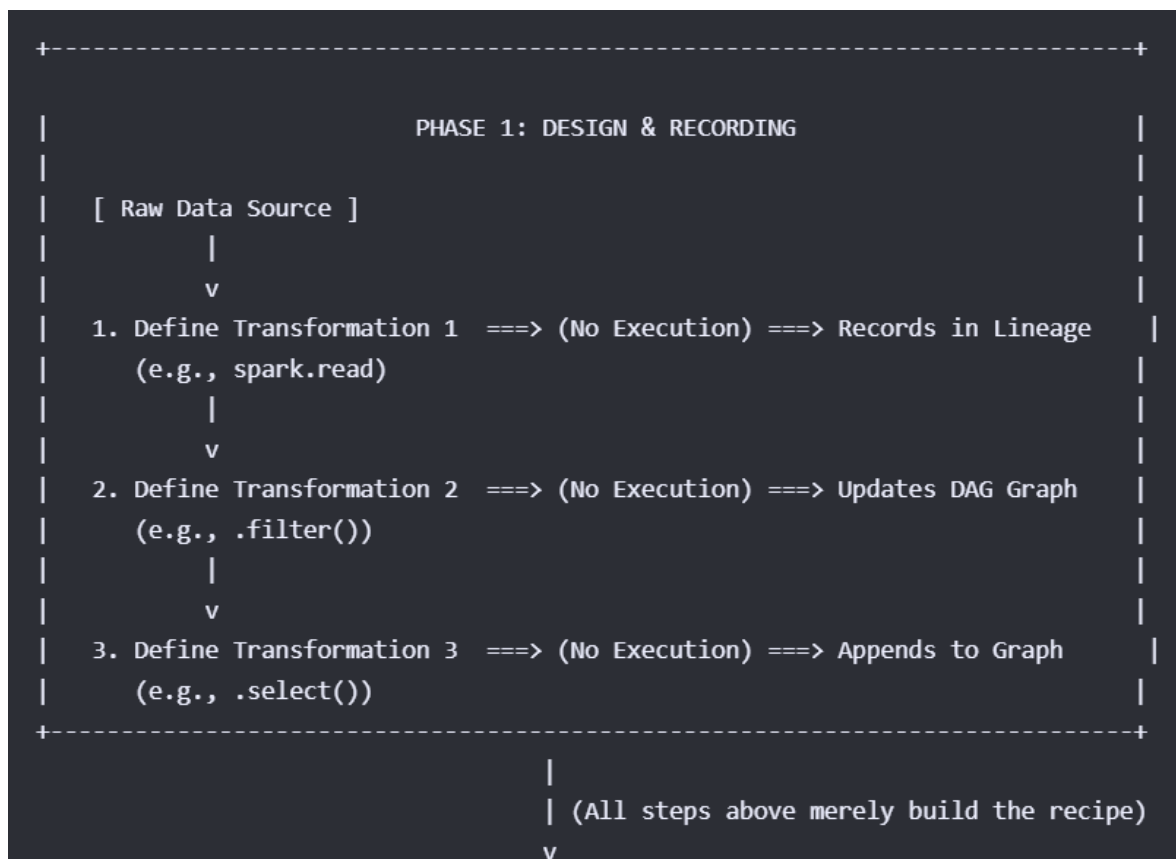
The execution flow remains idle until an action is called. An action tells Spark, *"I need the final answer now."* This request prompts Spark to stop recording and start computing.

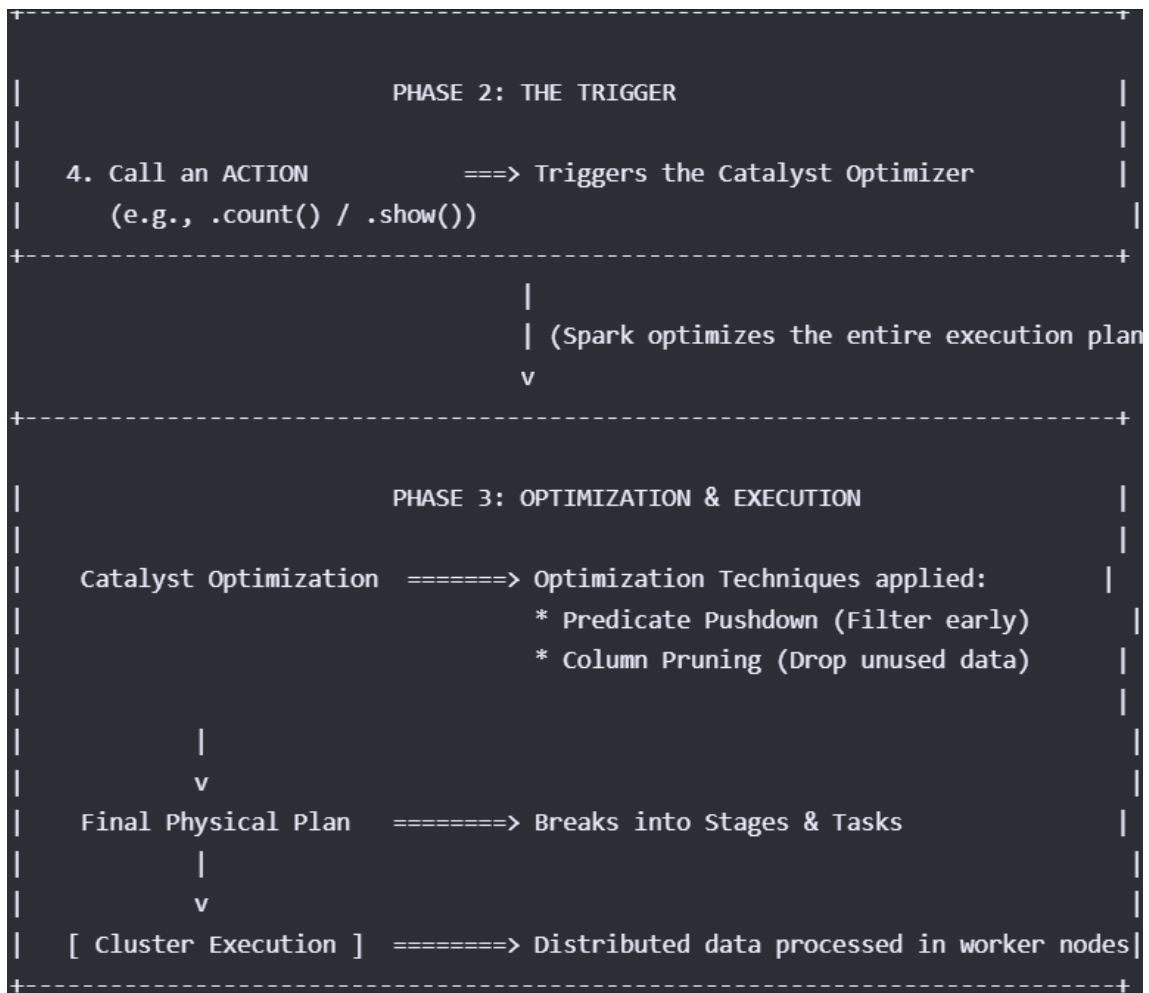
- **Examples:** `show()`, `count()`, `collect()`, `write()`, `saveAsTextFile()`.

3. Catalyst Optimizer (The Secret Sauce)

Because Spark waits until the action is triggered, the [Catalyst Optimizer Engine](#) evaluates the entire chain of transformations at once. It rearranges the workflow for maximum performance using two primary methods:

- **Predicate Pushdown:** If you filter data at step 5, Spark pushes that filter back to step 1 so it only reads the rows you actually need from the source database. [1]
- **Column Pruning:** If your source file has 100 columns but you only select 2, Spark updates the raw data reader to only load those 2 columns





Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, when, trim, lower, count, sum, avg, min, max
)
from pyspark.sql.types import (
    StructType, StructField, IntegerType,
    StringType, DoubleType
)

spark = SparkSession.builder \
    .appName("SparkDataFrameDataProcessing") \
    .master("local[*]") \
    .getOrCreate()

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("C:\\Users\\VIVOB00K PRO\\Downloads\\source.csv")
# df = spark.read.csv("C:\\Users\\VIVOB00K PRO\\Downloads\\source.csv", header=True, inferSchema=True)

print("Original Dataset")
df.show(5)

print("Schema:")
df.printSchema()
```

```
... Original Dataset
+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|category| price|base_price| region|priority|  status|amount|user_id|
+-----+-----+-----+-----+-----+-----+-----+-----+
|      1|Clothing| 14.75|    14.75|  South|   High|Completed| 53.72|  2424|
|      2| Sports| 25.57|    25.57|  South|   Low|Completed| 80.73|  9928|
|      3| Sports|118.02|   118.02| North|   High|Returned|263.07|  6514|
|      4|Clothing| 27.62|    27.62|  East|   Low|Cancelled|115.73|  9785|
|      5|Clothing|194.89|   194.89|Central|Medium|Cancelled| 628.5| 1750|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
Schema:
root
  |-- product_id: integer (nullable = true)
  |-- category: string (nullable = true)
  |-- price: double (nullable = true)
  |-- base_price: double (nullable = true)
  |-- region: string (nullable = true)
  |-- priority: string (nullable = true)
  |-- status: string (nullable = true)
  |-- amount: double (nullable = true)
  |-- user_id: integer (nullable = true)
```

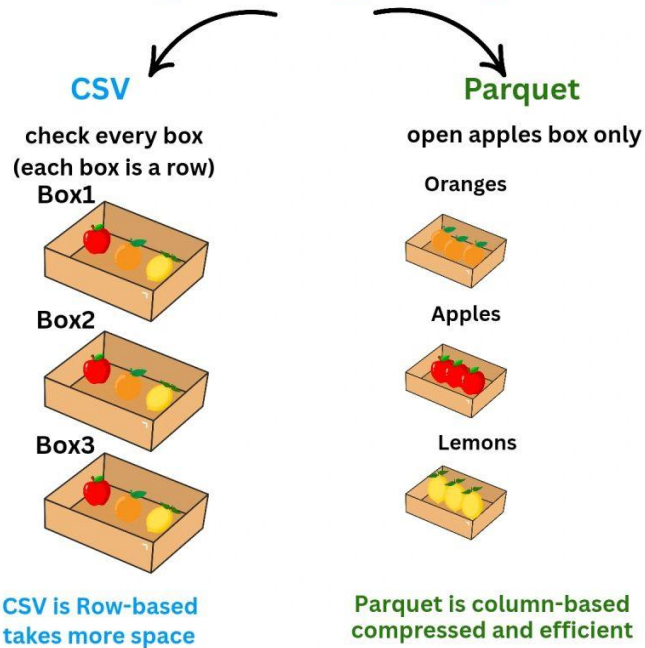
Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Direct Comparison

Feature	CSV (Comma-Separated Values)	Apache Parquet
Storage Layout	Row-based (data saved row by row)	Columnar (data saved column by column)
Data Format	Plain text (human-readable)	Binary (machine-readable)
File Size	Large (uncompressed by default)	Small (highly compressed)
Schema & Types	No embedded schema (all values are text)	Built-in schema (stores precise data types)
Query Speed	Slow (must scan the entire file)	Very fast (can skip irrelevant columns/rows)
Complex Structures	Flat tabular data only	Natively supports nested and array structures

CSV VS Parquet

If you want apples only



Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
df_display = df.select("product_id", "price").filter(df.category == "Electronics")
print("Filtering the Data on the basis of category ")
df_display.show(20)
```

[16] ✓ 1.7s

... Filtering the Data on the basis of category

product_id	price
9	1619.84
13	1376.55
14	1909.94
23	835.11
33	1352.33
37	1677.46
57	1720.63
60	1818.51
63	1901.74
77	1169.43
87	1538.06
103	682.53
116	643.51
118	1396.05
120	1584.25
123	389.78
130	1193.61
131	1938.38
138	448.63
149	928.52

only showing top 20 rows

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

```
from pyspark.sql.functions import col

df = df.withColumnRenamed("category", "product") \
        .withColumn("price", col("price").astype("double"))
print("Data after renaming and changing the data type")
df.show(5)
df.printSchema()
```

[23] ✓ 0.2s

... Data after renaming and changing the data type

product_id	product	price	base_price	region	priority	status	amount	user_id
1	Clothing	14.75	14.75	South	High	Completed	53.72	2424
2	Sports	25.57	25.57	South	Low	Completed	80.73	9928
3	Sports	118.02	118.02	North	High	Returned	263.07	6514
4	Clothing	27.62	27.62	East	Low	Cancelled	115.73	9785
5	Clothing	194.89	194.89	Central	Medium	Cancelled	628.5	1750

only showing top 5 rows

root

```
-- product_id: integer (nullable = true)
-- product: string (nullable = true)
-- price: double (nullable = true)
-- base_price: double (nullable = true)
-- region: string (nullable = true)
-- priority: string (nullable = true)
-- status: string (nullable = true)
-- amount: double (nullable = true)
-- user_id: integer (nullable = true)
```

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Normal execution: Spark reads data from a source, applies a chain of transformations (each transformation is *lazily* recorded, not executed immediately), and when an action is finally called, the whole chain runs, building the result RDD/DataFrame.

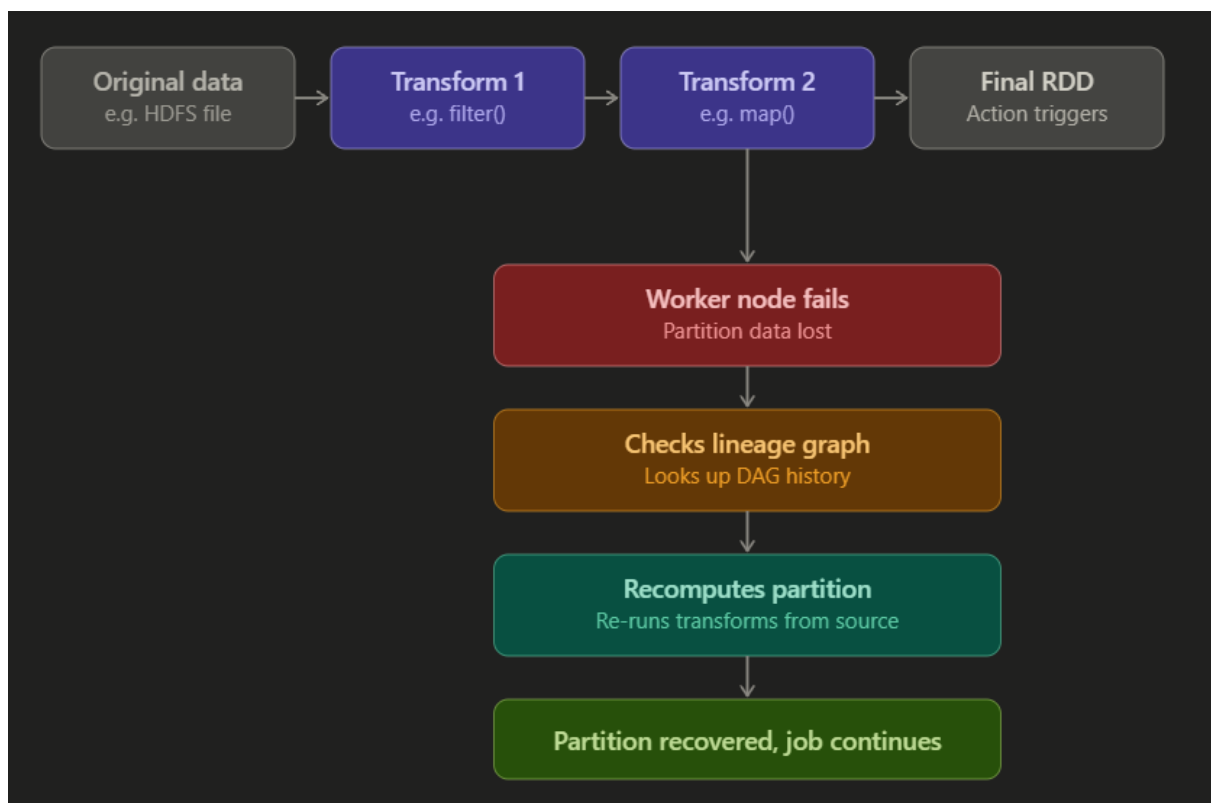
When a worker node fails mid-job:

1. Partition lost — the executor holding one or more partitions crashes, and that in-memory (or on-disk) data is gone.
2. Spark consults the lineage graph — since every RDD/DataFrame remembers exactly which transformations were applied to which parent, Spark can trace backward from the lost partition to find out precisely how it was derived.
3. Recomputation, not replication — Spark re-executes only the transformations needed to rebuild the missing partition(s), starting from the original data source (or from the nearest cached/checkpointed stage, if one exists closer than the source). It does not need to recompute the entire dataset — only the affected partitions.

4. Job continues transparently — once the partition is rebuilt, execution resumes as if nothing happened. The rest of the cluster keeps working on other partitions in parallel.

Why this beats data replication (used by traditional distributed systems like HDFS with 3x replication):

- No extra storage or network overhead spent copying every byte multiple times across nodes.
- Recovery cost is proportional only to the lost partition's transformation chain, not the whole dataset.
- This lineage-based, "recompute on failure" model is literally what the R in RDD (Resilient Distributed Dataset) stands for.



Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

```
logic_filter = df.filter((df.status == "Completed") & (df.amount > 1000))
print("Data filtering using the logical operator")
logic_filter.show()
```

[25] ✓ 1.2s

... Data filtering using the logical operator

product_id	product	price	base_price	region	priority	status	amount	user_id
15	Furniture	543.23	543.23	Central	Low	Completed	1801.12	2832
40	Furniture	873.74	873.74	North	Medium	Completed	3849.78	8541
57	Electronics	1720.63	1720.63	West	Medium	Completed	8189.36	7389
64	Furniture	1448.6	1448.6	East	High	Completed	5891.06	6096
81	Furniture	867.47	867.47	South	High	Completed	1264.28	8699
86	Furniture	2889.54	2889.54	Central	Low	Completed	14442.51	5415
101	Sports	477.53	477.53	West	Low	Completed	1620.42	5708
102	Furniture	2265.62	2265.62	North	Low	Completed	6034.37	1474
117	Sports	297.8	297.8	West	Low	Completed	1280.99	5583
121	Furniture	2457.85	2457.85	South	Low	Completed	4828.58	3129
123	Electronics	389.78	389.78	West	Low	Completed	1532.53	6931
130	Electronics	1193.61	1193.61	East	Low	Completed	3014.34	7929
142	Sports	423.8	423.8	Central	Medium	Completed	1043.49	6562
143	Furniture	1065.9	1065.9	Central	Low	Completed	3228.12	6724
149	Electronics	928.52	928.52	East	Low	Completed	2710.45	5059
236	Furniture	697.64	697.64	North	Medium	Completed	1275.2	NULL
312	Furniture	1535.17	1535.17	West	Low	Completed	4634.38	6916
318	Electronics	653.12	653.12	South	High	Completed	2464.34	3684
329	Electronics	1946.49	1946.49	East	High	Completed	9088.85	9515
330	Sports	353.41	353.41	North	Low	Completed	1032.95	7461

only showing top 20 rows

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory. Predicate pushdown in Parquet, explained

Parquet doesn't store data as one giant blob — it splits a file into row groups (chunks of rows, typically 128MB–1GB each). For every column in every row group, Parquet stores lightweight metadata in the file footer: min value, max value, and null count.

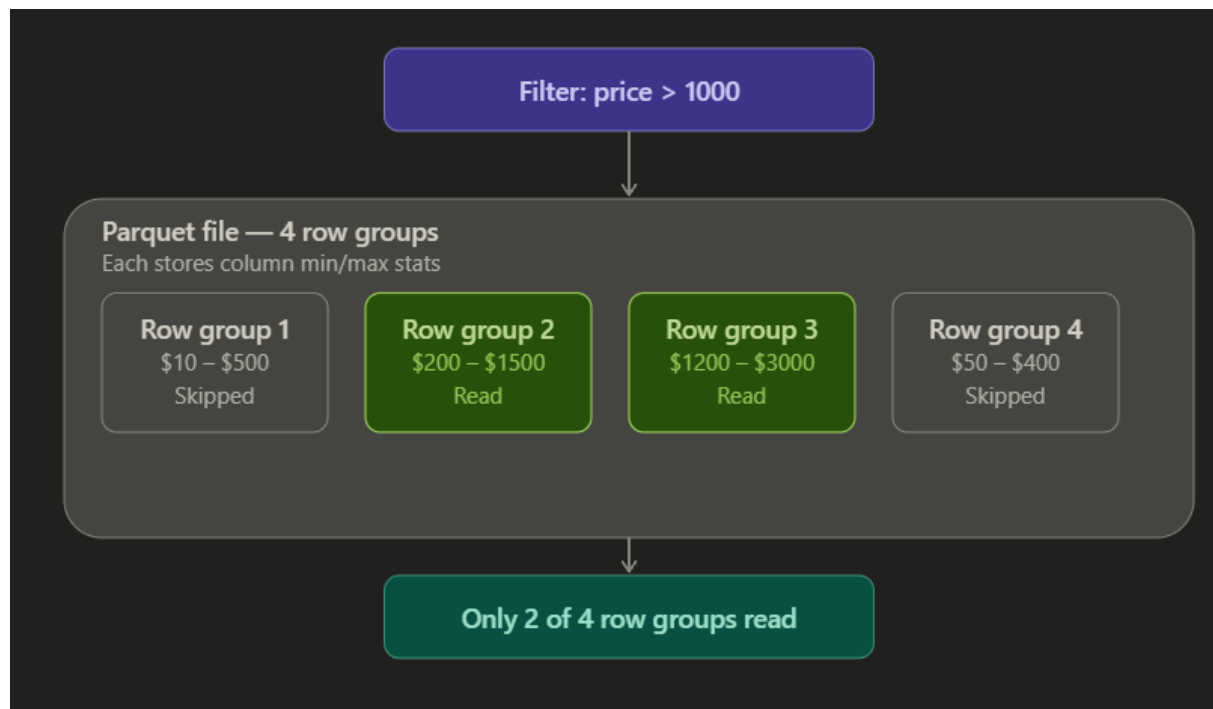
What happens when you apply a filter like `price > 1000`:

1. Spark reads only the footer/metadata first — this is tiny compared to the actual data.
2. For each row group, Spark compares your filter condition against that row group's min/max stats:
 - Row group 1 (min \$10, max \$500) → max is below 1000, so *no row in this group can possibly satisfy the filter* → entire row group is skipped, never read from disk.
 - Row group 2 (min \$200, max \$1500) → range overlaps the threshold → must be read (some rows may qualify).
 - Row group 3 (min \$1200, max \$3000) → fully above 1000 → read (all/most rows likely qualify).
 - Row group 4 (min \$50, max \$400) → max is below threshold → skipped.

- Only the row groups that *could* contain matching rows are actually pulled from disk into memory.

The compounding effect:

- Less disk I/O — skipped row groups are never even opened.
- Less memory pressure — only surviving row groups get decompressed and loaded.
- Combined with columnar storage, Spark can also skip irrelevant *columns* within the row groups it does read (this is projection pushdown, working alongside predicate pushdown) — so it reads only the columns and row groups it actually needs.



Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col

df_addcol = df.withColumn("final_price", col("base_price") * 1.18)
print("Data after adding the new column")
df_addcol.show()
```

[26] ✓ 0.5s

... Data after adding the new column

product_id	product	price	base_price	region	priority	status	amount	user_id	final_price
1	Clothing	14.75	14.75	South	High	Completed	53.72	2424	17.404999999999998
2	Sports	25.57	25.57	South	Low	Completed	80.73	9928	30.1726
3	Sports	118.02	118.02	North	High	Returned	263.07	6514	139.2636
4	Clothing	27.62	27.62	East	Low	Cancelled	115.73	9785	32.5916
5	Clothing	194.89	194.89	Central	Medium	Cancelled	628.5	1750	229.97019999999998
6	Furniture	2341.9	2341.9	South	High	Returned	4607.66	6977	2763.442
7	Grocery	38.28	38.28	East	Low	Completed	128.56	5010	45.1704
8	Grocery	47.3	47.3	Central	High	Cancelled	205.28	4752	55.813999999999999
9	Electronics	1619.84	1619.84	North	High	Cancelled	2742.44	8517	1911.4111999999998
10	Grocery	27.96	27.96	Central	Low	Cancelled	110.09	7543	32.9928
11	Books	17.06	17.06	Central	Medium	Completed	67.8	3504	20.130799999999997
12	Grocery	79.62	79.62	North	Medium	Returned	262.96	5119	93.9516
13	Electronics	1376.55	1376.55	Central	Medium	Cancelled	1746.18	8433	1624.329
14	Electronics	1909.94	1909.94	East	Low	Pending	5600.45	5889	2253.7291999999998
15	Furniture	543.23	543.23	Central	Low	Completed	1801.12	2832	641.0114
16	Books	53.33	53.33	East	High	Completed	96.62	2290	62.929399999999994
17	Clothing	149.06	149.06	Central	High	Pending	532.29	3705	175.89079999999998
18	Toys	81.51	81.51	South	Low	Pending	309.28	7118	96.1818
19	Beauty	135.44	135.44	South	High	Completed	300.67	4770	159.8192
20	Furniture	120.85	120.85	North	High	Completed	556.24	2160	142.60299999999998

only showing top 20 rows

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Transformations	Actions
Lazy — build a plan, don't execute	Eager — trigger actual execution
Return a new DataFrame/RDD	Return a value or write output
Examples: filter(), select(), withColumn(), groupBy(), join()	Examples: show(), collect(), count(), write()

Two examples of each:

- Transformations: filter(), select()
- Actions: count(), collect()

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```

• from pyspark.sql import SparkSession
  import os

  os.environ["HADOOP_HOME"] = "C:\\hadoop"
  os.environ["PATH"] = "C:\\hadoop\\bin;" + os.environ["PATH"]

  spark = SparkSession.builder.appName("Q12_ParquetFilterCSV").getOrCreate()

  df_csv = spark.read.option("header", True).option("inferSchema", True) \
    |.csv("C:\\Users\\VIVOBOK PRO\\Downloads\\source.csv")

  df_csv.write.mode("overwrite").parquet("data/source_parquet")

  df = spark.read.parquet("data/source_parquet")
  df_filtered = df.filter(df.user_id.isNotNull())
  df_filtered.write.mode("overwrite").option("header", True).csv("data/output_filtered")

  print("Total rows:", df.count())
  print("Filtered rows:", df_filtered.count())
  df_filtered.show(5)

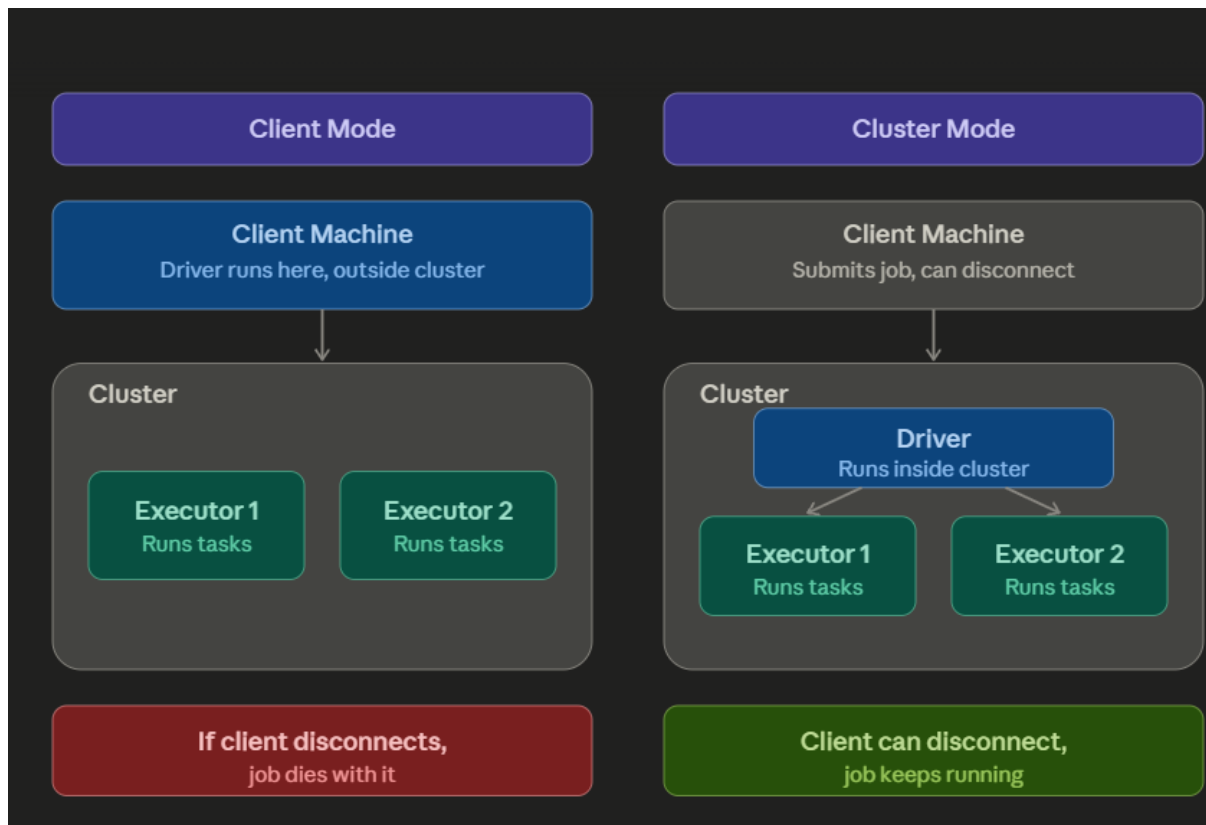
```

```

Total rows (df.count()):      8000
Filtered rows (df_filtered.count()): 7615

```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?



Client Mode:

The Driver process runs on the machine you submitted the job from — your laptop, a gateway/edge node, or wherever you ran `spark-submit`. This machine stays *outside* the cluster's resource management. The Driver here is responsible for building the DAG, scheduling tasks, and collecting results — and it does all of that while physically sitting outside the pool of worker nodes that the cluster manager controls.

- Good for: interactive work (Spark shell, Jupyter notebooks, debugging sessions) where you want to see logs and output live on your own screen.
- Risk: if your laptop sleeps, your network drops, or your SSH session to the edge node closes — the Driver process dies with it. Since the Driver coordinates everything, the entire job dies too, even if all the executors in the cluster are still healthy and running fine.

Cluster Mode:

The Driver process runs inside the cluster itself — the cluster manager (YARN, Kubernetes, etc.) allocates a container/node for the Driver, just like it does for executors. Once you submit the job, your local machine's only role was to hand off the request.

- Good for: production, scheduled batch jobs, anything unattended (cron jobs, Airflow-triggered pipelines, long-running ETL).
- Benefit: after submission, your client machine can disconnect, shut down, or move on to other work — the Driver keeps running inside the cluster, coordinating the executors independently. The job's fate is no longer tied to your local machine's uptime.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
df_logic = df.filter((df.region == "North") | (df.priority == "High"))
print("Data filtering using the logical AND operator")
df_logic.show()
```

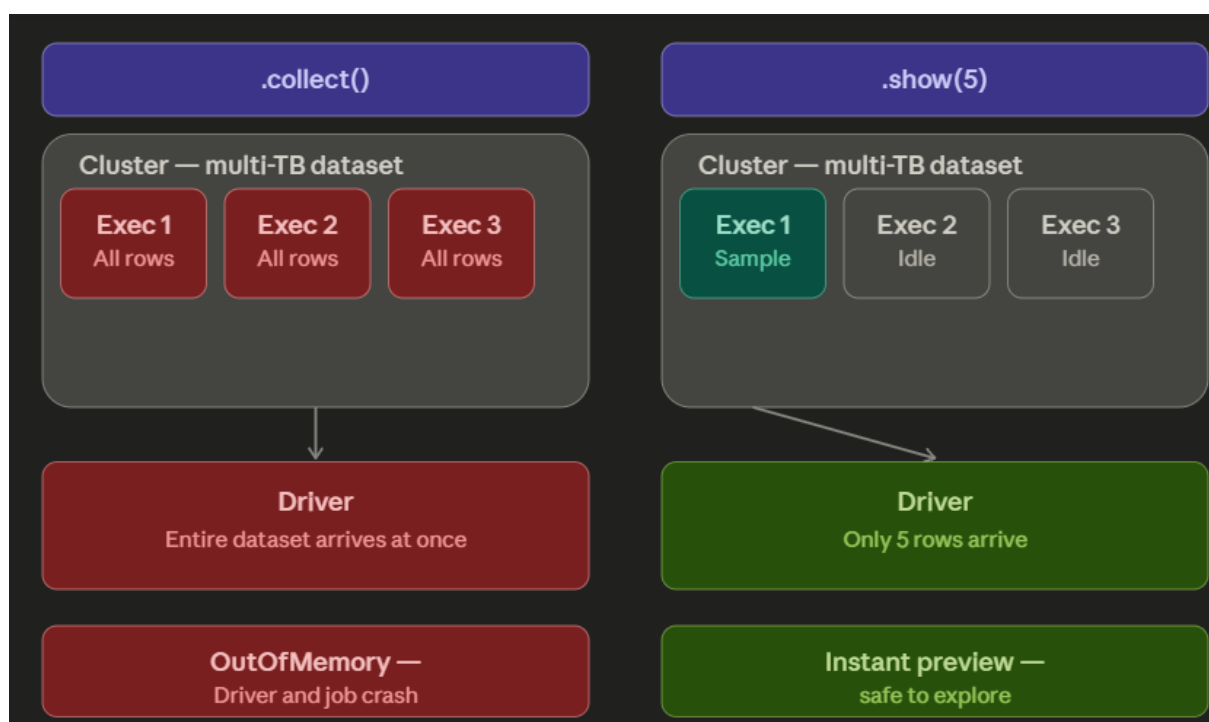
[7] ✓ 0.6s

... Data filtering using the logical AND operator

product_id	category	price	base_price	region	priority	status	amount	user_id
1	Clothing	14.75	14.75	South	High	Completed	53.72	2424
3	Sports	118.02	118.02	North	High	Returned	263.07	6514
6	Furniture	2341.9	2341.9	South	High	Returned	4607.66	6977
8	Grocery	47.3	47.3	Central	High	Cancelled	205.28	4752
9	Electronics	1619.84	1619.84	North	High	Cancelled	2742.44	8517
12	Grocery	79.62	79.62	North	Medium	Returned	262.96	5119
16	Books	53.33	53.33	East	High	Completed	96.62	2290
17	Clothing	149.06	149.06	Central	High	Pending	532.29	3705
19	Beauty	135.44	135.44	South	High	Completed	300.67	4770
20	Furniture	120.85	120.85	North	High	Completed	556.24	2160
21	Furniture	907.58	907.58	Central	High	Returned	1652.33	7669
23	Electronics	835.11	835.11	North	High	Pending	1335.24	7912
24	Grocery	29.3	29.3	North	Medium	Completed	29.67	1241
27	Grocery	20.61	20.61	North	Low	Completed	81.23	8811
28	Grocery	7.57	7.57	South	High	Completed	27.52	2964
29	Furniture	1778.87	1778.87	North	Medium	Cancelled	8406.52	6147
31	Beauty	95.07	95.07	North	High	Pending	278.05	6718
33	Electronics	1352.33	1352.33	North	High	Cancelled	1737.47	3546
34	Toys	45.85	45.85	East	High	Cancelled	134.01	1832
35	Clothing	130.51	130.51	North	High	Cancelled	527.09	5291

only showing top 20 rows

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset? i don't have a csv file with me how to create a csv file using llm



`.collect()` is an **action** that tells every executor across the cluster: *"send me your entire partition of data, right now."* Spark then gathers all of that data — every single row from every single partition — and serializes it all into the **Driver's memory**.

The problem: the Driver is typically **one machine** with a modest, fixed memory allocation (often just a few GB), while your dataset might be spread across **hundreds of executors holding terabytes combined**. Trying to cram a multi-terabyte dataset into one machine's memory space almost always ends in an **OutOfMemoryError**, crashing the Driver — and since the Driver coordinates the entire job, the whole application dies with it.

Why `.show(5)` is safe:

`.show(5)` only asks Spark for **5 rows** to display. Because of lazy evaluation and Spark's Catalyst optimizer, Spark is smart enough to apply **limit pushdown**:

- It doesn't need to compute the full dataset across all partitions first.
- It typically starts reading from just the **first partition** (or as few partitions as needed) until it finds 5 rows, then **stops** — the other executors may never even get triggered (they stay idle).
- Only those 5 rows get shipped back to the Driver, which is a negligible amount of memory regardless of how large the underlying dataset is.